

PROJECT REVIEW 1

Linux Signal-Based Process Communication and Control System

Operating Systems and Systems Programming | 25CS2104E | 2026-27, Term I

Team 20 • Section 13

Lahari Namala

2520090226

K. Keathana Vikashini

2520090236

C. Poojitha

2520090234

P. Bhagya Sri

2520080024

Abstract

The project proposes a Linux based control system in which **one parent process creates and manages three child worker processes.**

The parent uses **POSIX signals** to pause, resume, request status and stop the children — either **one child by its PID** or **all three together as a process group.**

Children report events back through an **anonymous pipe**, and a second terminal can send commands through a **named FIFO.**

The parent confirms each state from **/proc/[pid]/status** and **reaps every finished child, so no zombie process is left behind.**

1 parent + 3 children

One controller, three workers

6 POSIX signals

Pause, resume, status, work, stop

Pipe + FIFO

Events in, commands out

Zero zombies

Every child is reaped

Problem Statement and Motivation

THE PROBLEM

- **No direct control**
Once children are running, the program cannot pause, resume or stop them on demand.
- **No state visibility**
The parent cannot tell whether a child is running, sleeping or stopped.
- **Unsafe termination**
Killing a child abruptly gives it no chance to finish its work cleanly.
- **Zombie and orphan processes**
Children that exit without being reaped stay in the kernel process table.

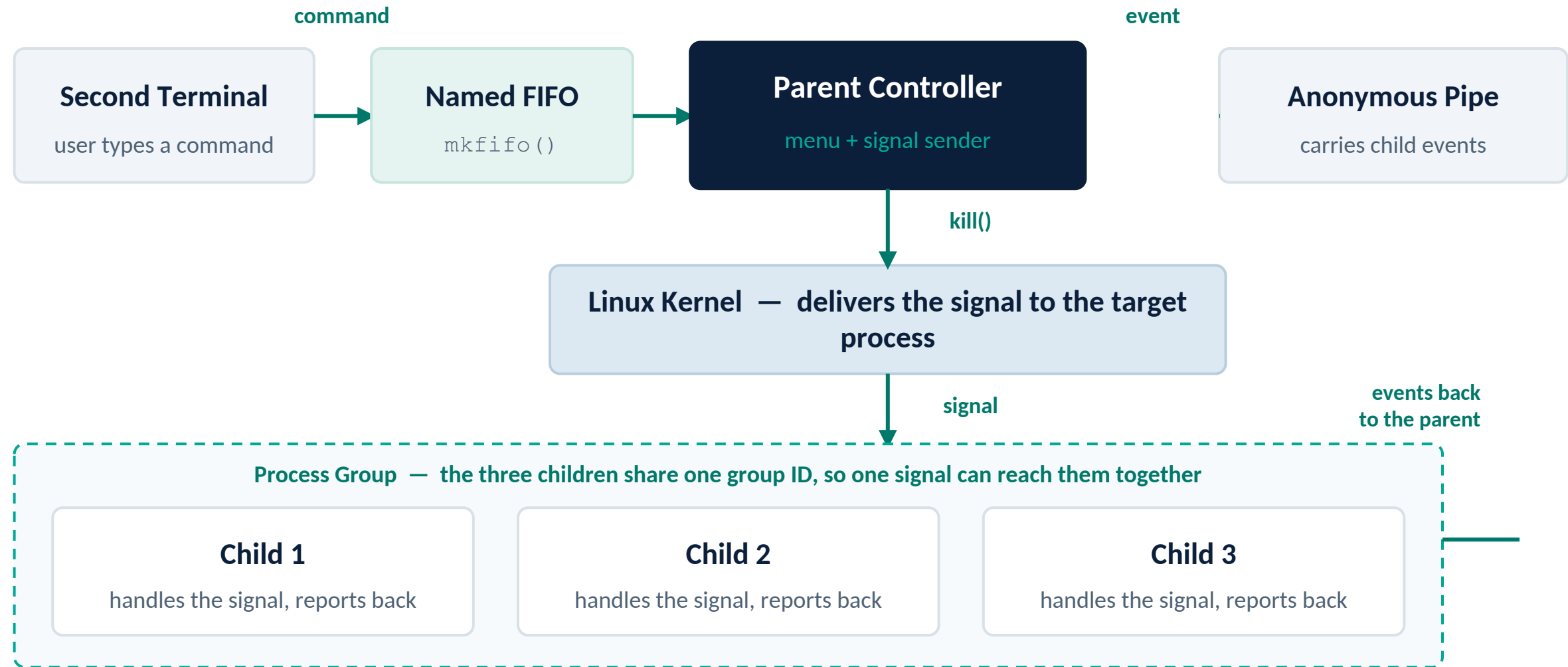
MOTIVATION

- **Signals are the standard way**
Linux itself controls running programs with signals — Ctrl+C is one.
- **Real software works like this**
Shells, web servers and system daemons manage their workers the same way.
- **Safe shutdown matters**
A polite stop request lets a process finish its work before it exits.
- **Strong learning value**
It connects processes, signals, pipes and kernel services in one project.

Objectives

- 1 Create the process family** Build a parent controller that creates **three child worker processes** using `fork()`, grouped together as one process group.
- 2 Control children two ways** Send a signal to **one child by its PID**, or to **all three at once** using the process group.
- 3 Use signals for every action** Apply **POSIX signals** to request status, trigger work, pause, resume and stop a child.
- 4 Exchange data both ways** Carry child events to the parent through a **pipe**, and outside commands into the parent through a **FIFO**.
- 5 Verify and clean up** Confirm each child state from `/proc/[pid]/status` and **reap every terminated child** so no zombie remains.

Proposed Architecture



How It Works — Step by Step

1 Program starts

The shell runs our program, which becomes the **parent process**.

2 Children are created

The parent calls **fork()** three times and puts the children in one process group.

3 Handlers are installed

Each child registers what to do for each signal using **sigaction()**.

4 Command is given

The user picks an action from the menu or writes it into the **FIFO**.

5 Signal is delivered

The parent calls **kill()** and the **kernel delivers the signal** to the target.

6 Result is verified

The child reports through the pipe; the parent checks the state and **reaps it when it exits**.

The same six steps repeat for every command the user gives.

Signals Used in the Project

Signal	From → To	Purpose	What the receiver does
SIGUSR1	Parent → Child	Ask the child for its current status	Runs its own handler and reports back
SIGUSR2	Parent → Child	Trigger one sample work event	Runs its own handler and reports back
SIGSTOP	Parent → Child	Pause the child	Kernel stops it — no handler allowed
SIGCONT	Parent → Child	Resume a stopped child	Kernel resumes it; child may note the event
SIGTERM	Parent → Child	Request a graceful exit	Finishes its work, then exits cleanly
SIGCHLD	Kernel → Parent	Tell the parent a child changed state	Parent calls waitpid() to check and reap

Important: **SIGSTOP** is enforced directly by the kernel — it cannot be caught, blocked or ignored, so no handler is written for it.

Linux Concepts and APIs Planned

Process Control

Creating the children and watching what happens to them.

`fork()`

`getpid()`

`setpgid()`

`waitpid()`

Communication

Moving events to the parent and commands into the parent.

`pipe()`

`mkfifo()`

`read()`

`write()`

`poll()`

Signal Programming

Deciding how a signal is handled and sending it safely.

`sigaction()`

`sigprocmask()`

`sigsuspend()`

`kill()`

Kernel Verification

Reading the state that the kernel itself maintains.

`/proc/[pid]/status`

Tools and Technologies

Ubuntu Linux

OPERATING SYSTEM

Provides the kernel, signals and the /proc file system.

C with GCC

LANGUAGE AND COMPILER

Standard language for calling Linux system calls directly.

POSIX API

SYSTEM INTERFACE

Portable set of process, signal and IPC functions.

Linux Terminal

TEST ENVIRONMENT

Runs the program and sends FIFO commands from a second window.

VS Code / vim

EDITOR

Writing and organising the project source files.

Git and GitHub

VERSION CONTROL

Tracking progress and submitting the project repository.

Repository: github.com/lahari2007namala-creator/2520090226_OSSP/tree/main/PROJECT

Expected Outcome



Menu-driven controller

A simple text menu to choose the child and the action.



Pause, resume and status

Children can be stopped, continued and asked for their state.



Live event reporting

Child events reach the parent through the pipe as they happen.



Verified kernel state

Each state confirmed from the kernel's /proc file.



Individual and group control

Signal one child by PID, or all three at once by process group.



Graceful termination

A child finishes its work before exiting on request.



External FIFO commands

A second terminal can control the system without stopping it.



Clean exit, no zombies

Every terminated child is reaped by the parent.

All items listed above are targeted outcomes — implementation begins after this review.

Implementation Plan



Each stage is tested before the next one begins, so the system grows step by step.

Conclusion

The proposed system demonstrates how Linux processes **communicate and change state** through **POSIX signals, pipes, FIFOs, process groups and kernel services**.

Team 20

Section 13

Review 1

Proposal stage

Next step

Begin Stage 1

Repository: github.com/lahari2007namala-creator/2520090226_OSSP/tree/main/PROJECT

Questions?